New chat
My stuff
Gems
Chats
A peak element is an element that is ...
Google Phone Call Recording Location
CMF Phone Screen Step On Options
Git vs. GitHub Explained
Google Phone Call Recording Setting...
How to Record All Calls
Settings and help

Binary Search Logic

Every Binary method in use While loop

A peak element is an element that is strictly greater than its neighbors.
Given a 0-indexed integer array nums, find a peak element, and return its index. If the array contains multiple peaks, return the index to any of the peaks.
You may imagine that nums[-1] = nums[n] = -∞. In other words, an element is always considered to be strictly greater than a neighbor that is outside the array.
You must write an algorithm that runs in O(log n) time.

use of javascript Binary search method and understand me in "Gujarati" Language

Show thinking

કોડમાં, આ સોલ્યુશન દ્વિપક્ષી શોધની **Binary Search** પદ્ધતિનો ઉપયોગ કરીને JavaScript માં લખવામાં આવ્યું છે. આ પદ્ધતિ અત્યંત ઝડપી છે કારણ કે તે દરેક તત્ત્વ તપાસવાને બદલે અડધા ઇનપુટને નકારી કાઢે છે. આ પદ્ધતિ ખાતરી કરે છે કે તે ઓછામાં ઓછા O(log n) સમયમાં પરિણામ આપવા માટે Binary Search-નો ઉપયોગ કરી શકીએ છીએ.

1. સ્થાનિક ઉકેલ (Local Solution)

શ્રેષ્ઠતમ ઉકેલ Binary Search સાથે (ઝડપ) કે ઉકેલ કમાં વધારેમાં વધારે પગલાં ભરે છે. પણ આવી ત્રણ સંભવિત સ્થિતિઓ માટે, આપણે O(log n) સમયમાં જવાબ આપવા માટે Binary Search-નો ઉપયોગ કરી શકીએ છીએ.

ઉકેલમાં સ્થાનિક ઉકેલ પદ્ધતિ (Mountain) કરીને કામ કરવામાં આવે છે:

- જો આપણે ઉકેલ **ઉપરની તરફ (Upward slope)** પર હોઈએ, તો જ્યાં સુધીમાં આપણે પહોંચી ગયા નથી.
- જો આપણે ઉકેલ **ઉતરતા (Downward slope)** પર હોઈએ, તો જ્યાં સુધીમાં આપણે પહોંચી ગયા નથી.

ઉદાહરણ:

1. કોઈપણ અનુક્રમ્ય અનુક્રિત્ય (`mid`) શોધો.
2. જો `nums[mid]` તેના ડાબેના તત્ત્વ કરતાં મોટું હોય, તો તેનો અર્થ એ છે કે તે ઉપર ઉતરી જવાની છે. તેથી ઉકેલ જમણી બાજુ (`right` સુધી) હશે.
3. જો `nums[mid]` તેના ડાબેના તત્ત્વ કરતાં `nums[mid+1]` કરતાં મોટું હોય, તો તેનો અર્થ એ છે કે આપણે ઉતરી ગયા છીએ અને તેના જમણેના તત્ત્વ કરતાં મોટું. તેથી ઉકેલ ડાબી બાજુ (`left` સુધી) અથવા `mid` પર હશે.

2. JavaScript Code Solution

અહીં JavaScript માં આ અલ્ગોરિધમનું રીમ્પ્લીમેન્ટેશન છે:

Javascript

```
/**
 * @param {number[]} nums
 * @return {number}
 */
var findPeakElement = function(nums) {
    let left = 0;
    let right = nums.length - 1;

    while (left < right) {
        // મધ્યમ બિંદુ ગણતરી કરવામાં આવી રહી છે
        let mid = Math.floor((left + right) / 2);

        // શું મધ્ય કરતાં જમણી બાજુ પર ઢોળાવ છે?
        // જો હા, તો પીક જમણી બાજુ હોવાની શક્યતા છે.
        if (nums[mid] < nums[mid + 1]) {
            left = mid + 1;
        } else {
            // જો આ સાચો છે તો ઢોળાવ કાં તો ડાબેની બાજુ હશે અથવા આપણે જ છીએ.
            // જો આપણે ત્યાંથી જઈશું તો આપણે ગુમાવીશું.
            right = mid;
        }
    }

    // જ્યારે લૂપ પૂરો થાય, ત્યારે left અને right એકસાથે મળે છે જ્યાં પીક હોવાની ખાતરી છે.
    return left;
};
```

3. ઉદાહરણ સાથે સમજૂતી (Step-by-Step Explanation)

ચાલો કહે `nums = [1, 2, 3, 1]` છે.

1. **શરૂઆત:**
   - `left = 0`, `right = 3`
   - `mid = Math.floor((0 + 3) / 2) = 1`
   - `nums[1]` (2 છે) અને `nums[2]` (3 છે) ની સરખામણી કરો.
2. **પહેલું પગલું:**
   - અહીં `nums[1] < nums[2]` (2 < 3) સાચું છે.
   - આનો અર્થ અમારો "ઢાળ" પર ઉપર જવું. પીક જમણી બાજુ છે.
   - તેથી, `left = mid + 1 = 2`.
3. **બીજું પગલું:**
   - હવે `left = 2`, `right = 3`.
   - નવું `mid = Math.floor(2 + 3) / 2) = 2`.
   - `nums[2]` (3 છે) અને `nums[3]` (1 છે) ની સરખામણી કરો.
4. **ત્રીજું પગલું:**
   - અહીં `nums[2] > nums[3]` (3 > 1) સાચું છે (1 નકાર છે).
   - આનો અર્થ અમારો "ઉતરાણ" પર હોવાનો અથવા પીક પર હોવાનો.
   - તેથી `right = mid = 2`.
5. **અંત:**
   - હવે `left = 2` અને `right = 2`.
   - Condition: `left < right` ખોટું છે તે લૂપ બહાર આવી ગયો.
   - જવાબ 2 (અનુક્રમ 2 અંદર 2 સેટ) પરત કરો.

Complexity Analysis

- **Time Complexity:** O(log n) - કારણ કે અમારો દરેક લૂપમાં સર્ચ સ્પેસને અડધો કરીએ છીએ.
- **Space Complexity:** O(1) - અમારો કોડ ચલવવાને સ્થિરતા જરૂરી છે.

શું તમે ઇચ્છો છો કે હું તેનો બીજો ઇષ્ટતમ (complex) ઉકેલ પણ આપું જ્યાં તેને "Dry Run" કરીને ખાતરી કરાવું?

You are Given an array Sorted in ascending order. Find the Starting and ending Position of a given Target value.

If target is not found in the array, return [-1, -1].

You must write an algorithm with O(log n) runtime complexity.

use of javascript Binary search and understand me in Gujarati language

Show thinking

કોડમાં, આ સોલ્યુશન દ્વિપક્ષી શોધ અથવા **Binary Search** નો ઉપયોગ કરીને કરવામાં આવ્યું છે.

અહીં અમારાને ટાર્ગેટ ઇનપુટની **શરૂઆતની (Starting)** અને **અંતિમ (Ending)** સ્થિતિમાં શોધવાની છે. આ અહીં અમારાને O(log n) સમય નિયંત્રણ કરીને કરવામાં આવે છે.

1. સ્થાનિક ઉકેલ (Local Solution)

શ્રેષ્ઠતમ Binary Search ના ભિન્નતા અમારાને `target` બાબત મળ્યા પછીને અટકાવી દેવાને બદલે ડાબી અને જમણી બાજુ સર્ચ કરવાની જરૂર પડે છે. આ અમારાને ઓછામાં ઓછા સમયમાં ઉકેલ પર પહોંચવાનું નથી.

અમારાએ આ કામ કે માટેના બે અલગ લૂપ્સ:

1. **First Occurrence (પ્રથમ ઘટના) શોધવું:**
   - અમારો `nums[mid] >= target` નો ઉપયોગ કરો. અમારો અહીં કે ઉમેદવાર કમનસીબી જ્યાંથી શરૂ થાય છે.
   - પછી અમારો તમે પણ કરશો **જ્યારે (left < right)** તમારાને ખાતરી કરવાનું કે દરેક વસ્તુ કે જ્યાં સુધી તમારું ઉકેલ પહોંચ્યા પછી અટકી જશે.
2. **Last Occurrence (છેલ્લી ઘટના) શોધવું:**
   - ફરીથી Binary Search કરો.
   - અમારો `nums[mid] <= target` નો ઉપયોગ કરો. તમારો કેટલો અહીં.
   - આ વખતે અમારો જમણી બાજુ **(Right Side)** તમારા માટે કામ કરવું જરૂરી. કારણ કે દરેક અહીં કે આ ઝડપિત તમારા ઉકેલ પછી પણ હોય.

2. JavaScript Code Solution

અહીં JavaScript માં આ અલ્ગોરિધમનું રીમ્પ્લીમેન્ટેશન છે:

Javascript

```
/**
 * @param {number[]} nums
 * @param {number} target
 * @return {number[]}
 */
var searchRange = function(nums, target) {

    // પ્રથમ અને અંતિમ ઘટનાઓને સર્ચ કરવાના માટેનું કાર્ય
    const findFirstPosition = (nums, target) => {
        let left = 0;
        let right = nums.length - 1;
        let firstPos = -1; // જો તે નહીં હોય તો ડિફોલ્ટ -1

        while (left <= right) {
            let mid = Math.floor(left + right) / 2);

            // મધ્ય તપાસો: જો આપણે લક્ષ્ય શોધી રહ્યા છીએ
            if (nums[mid] >= target) {
                firstPos = mid; // તે જ્યાં પણ હશે તેને સાચવો
                right = mid - 1; // ડાબી બાજુ તપાસો
            } else if (nums[mid] < target) {
                left = mid + 1;
            } else {
                return mid;
            }
        }
        return firstPos;
    };

    // છેલ્લી ઘટના શોધવાના ફંક્શન માટેનું કાર્ય
    const findLastPosition = (nums, target) => {
        let left = 0;
        let right = nums.length - 1;
        let lastPos = -1;

        while (left <= right) {
            let mid = Math.floor(left + right) / 2);

            if (nums[mid] === target) {
                lastPos = mid; // તે જ્યાં પણ હશે તેને સાચવો
                left = mid + 1; // જમણી બાજુ તપાસો
            } else if (nums[mid] < target) {
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }
        return lastPos;
    };

    const start = findFirstPosition(nums, target);
    const end = findLastPosition(nums, target);

    return [start, end];
};
```

3. ઉદાહરણ સાથે સમજૂતી (Step-by-Step Explanation)

ચાલો કહે `nums = [5, 7, 7, 8, 8, 10]` અને `target = 8` છે.

Step 1: પ્રથમ પોઝિશન શોધવી (`findFirstPosition`)

- `binary search` પણ `mid` પર ચલાવો.
- અમારો `firstPos` ને 4 ઉમેદવાર તરીકે સેટ કરો.
- હવે અમારો કામની **જમણી બાજુ** (`right = mid - 1`) શોધવાનું છે કે શું ત્યાંના પણ આવે છે?
- તેમાંથી અમારો `lastPos` ને અમારો ત્યાંનો સ્થિતિ તરીકે સેટ કરો.

Step 2: અંતિમ પોઝિશન શોધવી (`findLastPosition`)

- ફરીથી binary search ચલાવો, `mid` પર ધ્યાન આપો.
- અમારો `lastPos` ને 4 તરીકેના તરીકે સેટ કરો.
- હવે અમારો જમણી બાજુ (`left = mid + 1`) જમણેનું શું છે તે પણ ચકાસવા માટે છે?
- તેમાંથી અમારો `lastPos` ને અમારો ત્યાંનો સ્થિતિ તરીકે સેટ કરો.

પરિણામ: `[4, 4]`

4. Complexity Analysis

- **Time Complexity:** O(log n) - અમારો બે વાર Binary Search કરીએ છીએ, એટલે O(log n) + O(log n) = O(log n).
- **Space Complexity:** O(1) - અમારો કોઈ મોટા ચોરસમાનમાં જવાબને સ્થિતિમાં કરીએ છીએ.

There is an integer array nums sorted in ascending order (with distinct values).
Prior to being passed to your function, nums is possibly rotated at an unknown pivot index k (1 <= k < nums.length) such that the resulting array is [nums[k], nums[k+1], ..., nums[n-1], nums[0], nums[1], ..., nums[k-1]] (0-indexed).
For example, [0,1,2,4,5,6,7] might be rotated at pivot index 3 and become [4,5,6,7,0,1,2].

Given the array nums after the possible rotation and an integer target, return the index of target if it is in nums, or -1 if it is not in nums.
You must write an algorithm with O(log n) runtime complexity.

Show thinking

કોડમાં, આ સોલ્યુશન મોડિફાઈ કરેલ **Binary Search** નો ઉપયોગ કરીને ઓપ્ટિમાઈઝ્ડ ટાઈમ કોમ્પ્લેક્સિટી હેઠળ કરવામાં આવે છે.

અહીં ફુલ્લેરી તમે છે કે મોટે ભાગનાં (ઉદાહરણ) તે તફાવત કે, તે સૂચવવામાં મદદગાર કામમાં આવે છે. દા.ત. `[4, 5, 6, 7, 0, 1, 2]`.

પણ, અમારા મોટા પરિણામ છે: **જો અમારાથી આ સર્ચમાં મધ્ય ભાગ કરીએ છીએ, તો ડાબેની બાજુ મધ્ય કરેલાં સંપૂર્ણપણે સોર્ટેડ (Sorted) હશે.** અમારાથી આ ખાતરીપૂર્વક જ કરવાની રહેવાનું.

1. સ્થાનિક ઉકેલ (Local Solution)

અમારાથી `mid` નક્કી કરવાનું કામ અને અહીં તપાસ કે તે મધ્યમાં ડાબેની બાજુ મધ્ય કરેલાં છે:

1. `mid` `nums[left] >= nums[mid]`, તો તમારે મધ્ય કરેલાં જમણી બાજુ સર્ચ કરવાની છે.
   - તેને જોઈ શકાતું જ્યારે `target` આ ડાબા અંતરમાં ઉમેરવામાં આવે છે? (`target >= nums[left]` અને `target < nums[mid]`)
   - જો હા, તો અમારો જમણી બાજુને બાકાત કરવું (`right = mid - 1`).
   - જો ના, તો તમારે અમારાથી આગળ જવું જોઈએ જમણેમાં (`left = mid + 1`).
2. `mid` `nums[left] < nums[mid]`, તો અમારાથી ડાબી બાજુ મધ્ય (અથવા તો તદ્દન) સોર્ટેડ કરે છે.
   - તેને જોઈ શકાતું જ્યારે `target` આ અમારાથી ડાબેમાં અને `mid` વચ્ચે છે? (`target >= nums[left]` અને `target < nums[mid]`)
   - જો હા, તો અમારાથી ડાબી બાજુને અંતરમાં (`right = mid - 1`).
   - જો ના, તો તમારે અમારાથી આગળ જવું જોઈએ જમણેમાં (`left = mid + 1`).

2. JavaScript Code Solution

Javascript

```
/**
 * @param {number[]} nums
 * @param {number} target
 * @return {number}
 */
var search = function(nums, target) {
    let left = 0;
    let right = nums.length - 1;

    while (left <= right) {
        let mid = Math.floor((left + right) / 2);

        // જો સર્ચિંગ જ પર હશે અહીં થશે
        if (nums[mid] === target) {
            return mid;
        }

        // કઈ બાજુ સૂર્ય અને સોર્ટેડ છે?
        if (nums[left] <= nums[mid]) {
            // જો આપણે ડાબી બાજુ છીએ
            if (nums[left] <= target && target < nums[mid]) {
                right = mid - 1; // ડાબી બાજુ શોધો
            } else {
                left = mid + 1;  // જમણી બાજુ શોધો
            }
        } else {
            // અન્યથા, જમણી બાજુ સોર્ટેડ છે
            if (nums[mid] < target && target <= nums[right]) {
                left = mid + 1; // જમણી બાજુ શોધો
            } else {
                right = mid - 1; // ડાબી બાજુ શોધો
            }
        }
    }

    // જો લૂપ તૂટ્યો તો -1 પરત કરો
    return -1;
};
```

3. ઉદાહરણ સાથે સમજૂતી (Dry Run)

ચાલો કહે `nums = [4, 5, 6, 7, 0, 1, 2]` અને `target = 0` છે.

1. **શરૂઆત:** `left = 0`, `right = 6`.
   - `mid = 3` (વેલ્યુ 7).
   - `nums[left] (4) <= nums[mid] (7)` → ડાબી બાજુ સોર્ટેડ છે.
   - શું તે કહી શું `target (0)` તે અને 4 ની વચ્ચે છે? ના.
   - તેથી, `left = mid + 1 = 4`.
2. **બીજું પગલું:** `left = 4`, `right = 6`. (સોર્ટેડ મધ્ય: `[0, 1, 2]`)
   - `mid = 5` (વેલ્યુ 1).
   - `nums[left] (0) <= nums[mid] (1)` → ડાબી બાજુ સોર્ટેડ છે.
   - શું તે કહી શું `target (0)` તે અને 0 ની વચ્ચે છે? હા.
   - તેથી, `right = mid - 1 = 4`.
3. **ત્રીજું પગલું:** `left = 4`, `right = 4`.
   - `mid = 4` (વેલ્યુ 0).
   - `nums[mid] == target` (0 == 0). અમારી મદદ!
   - **return 4.**

Complexity Analysis

- **Time Complexity:** O(log n) - કારણ કે અમારો Binary Search યથાવતી લીધે.
- **Space Complexity:** O(1) - અમારો ચલાવવી પેઈડી પ્રમાણમાં નથી.

શું તમને આ "Rotated" સંબંધિત સમસ્યાઓ?

Gemini can make mistakes, so double-check it